What is an Aggregate Function?

An **aggregate function** takes **multiple rows** and returns **a single value (summary)**.

🎯 **Simple Example**

Table: sales

amount
100
200
300

SELECT SUM(amount) FROM sales;

👉 Output:

600

✓ Many rows → one value

📊 **Main Aggregate Functions (You MUST know)**

1. COUNT()

Counts rows

SELECT COUNT(*) FROM sales;

👉 counts all rows

---

2. SUM()

Total of values

SELECT SUM(amount) FROM sales;

---

3. AVG()

Average value

SELECT AVG(amount) FROM sales;

---

4. MIN()

Smallest value

SELECT MIN(amount) FROM sales;

---

5. MAX()

Largest value

SELECT MAX(amount) FROM sales;

🧠 **Key Behavior (Very Important)**

👉 **Aggregate ignores NULL**

amount

100

NULL

200

SELECT AVG(amount) FROM sales;

👉 Result:

150

✓ NULL is ignored

🔥 **Aggregate + GROUP BY (Real Power)**

Without GROUP BY → whole table is one group

With GROUP BY → multiple groups

Example

SELECT category, SUM(amount)

FROM sales

GROUP BY category;

👉 Output:

category	sum
Electronics	3000
Furniture	800

⚠️ Golden Rule

You cannot mix:

❌ Normal column + aggregate
WITHOUT GROUP BY

❌ Wrong

SELECT category, SUM(amount) FROM sales;

👉 Error (which category??)

✅ Correct

SELECT category, SUM(amount)

FROM sales

GROUP BY category;

🔥 HAVING (Advanced Filtering)

WHERE → filters rows
HAVING → filters groups

Example

SELECT category, SUM(amount) AS total

FROM sales

GROUP BY category

HAVING total > 1000;

⚙️ Execution Flow with Aggregates

FROM → WHERE → GROUP BY → HAVING → SELECT

🔍 **Real Use Cases**

- Total revenue → SUM()
- Number of users → COUNT()
- Average salary → AVG()
- Highest sale → MAX()

⚠️ **Common Mistakes (Interview Traps)**

❌ **Mistake 1: Using aggregate in WHERE**

SELECT * FROM sales WHERE SUM(amount) > 1000;

👉 ❌ Invalid

✓ Fix:

HAVING SUM(amount) > 1000;

❌ **Mistake 2: Forgetting GROUP BY**

❌ **Mistake 3: Misunderstanding COUNT**

COUNT(*) → counts all rows

COUNT(column) → ignores NULL

COUNT(DISTINCT column) → unique count

🔥 **Advanced Examples**

1. Count unique customers

SELECT COUNT(DISTINCT customer_id) FROM sales;

---

2. Total sales per city

SELECT store_location, SUM(amount)

FROM sales

GROUP BY store_location;

---

3. Highest sale per category

```
SELECT category, MAX(amount)
FROM sales
GROUP BY category;
```

🧠 **Mental Model**

Aggregate = “summarize data”

⚡ **Interview Insights**

💡 **Q: Difference between WHERE and HAVING?**

WHERE	HAVING
filters rows	filters groups
before aggregation	after aggregation

💡 **Q: Can we use alias in HAVING?**

```
SELECT category, SUM(amount) AS total
FROM sales
GROUP BY category
HAVING total > 1000;
```

✓ Yes (MySQL allows it)

Interview-level problems + complex aggregation patterns:

Use your sales table.

🔥 **PROBLEM 1: Top Revenue Category**

❓ **Question**

Find the **category with highest total revenue**

✅ **Solution**

```
SELECT category, SUM(amount) AS total_revenue
FROM sales
GROUP BY category
ORDER BY total_revenue DESC
LIMIT 1;
```

🧠 **Insight**

- Aggregate first (SUM)
- Then sort
- Then limit

🔥 **PROBLEM 2: Revenue per City (Only big cities)**

❓ **Question**

Show cities where total revenue > 2000

✅ **Solution**

```
SELECT store_location, SUM(amount) AS total_revenue
FROM sales
GROUP BY store_location
HAVING total_revenue > 2000;
```

🧠 **Key Concept**

👉 HAVING filters **after aggregation**

🔥 **PROBLEM 3: Duplicate Customers**

❓ **Question**

Find customers who made more than 1 purchase

✅ **Solution**

```
SELECT customer_id, COUNT(*) AS total_orders
FROM sales
GROUP BY customer_id
HAVING COUNT(*) > 1;
```

🔥 **PROBLEM 4: Average Order Value per Category**

❓ **Question**

Find average sale amount per category

✅ **Solution**

```
SELECT category, AVG(amount) AS avg_sale
FROM sales
GROUP BY category;
```

🔥 **PROBLEM 5: Highest Sale per Location**

❓ **Question**

Find max sale in each city

✅ **Solution**

```
SELECT store_location, MAX(amount) AS max_sale
FROM sales
GROUP BY store_location;
```

🔥 **PROBLEM 6 (INTERVIEW FAVORITE)**

❓ **Question**

Find **second highest sale amount**

✅ **Solution**

```
SELECT DISTINCT amount
FROM sales
ORDER BY amount DESC
LIMIT 1,1;
```

🧠 **Why it works**

- DISTINCT removes duplicates
- LIMIT skips first (highest), picks second

🔥 **PROBLEM 7: Total Revenue per Day**

❓ **Question**

Daily sales summary

✅ **Solution**

```
SELECT sale_date, SUM(amount) AS total_revenue
FROM sales
GROUP BY sale_date
ORDER BY sale_date;
```

🔥 **PROBLEM 8: Category Contribution %**

❓ **Question**

What % of total revenue comes from each category?

✅ **Solution**

```
SELECT
  category,
  SUM(amount) AS category_total,
  (SUM(amount) / (SELECT SUM(amount) FROM sales)) * 100 AS percentage
FROM sales
```

GROUP BY category;

🧠 **Advanced Concept**

👉 Subquery inside aggregation

🔥 **PROBLEM 9 (TOUGH)**

❓ **Question**

Find customers whose total purchase > average total purchase

✅ **Solution**

```
SELECT customer_id, SUM(amount) AS total_spent
FROM sales
GROUP BY customer_id
HAVING total_spent > (
  SELECT AVG(total)
  FROM (
    SELECT SUM(amount) AS total
    FROM sales
    GROUP BY customer_id
  ) t
);
```

🧠 **Insight**

- Nested aggregation
- Very common in interviews

🔥 **PROBLEM 10 (REAL-WORLD)**

❓ **Question**

Top 2 products by total sales amount

✅ **Solution**

```
SELECT product, SUM(amount) AS total_sales
FROM sales
GROUP BY product
ORDER BY total_sales DESC
LIMIT 2;
```

⚠️ **TRICKY INTERVIEW QUESTIONS**

❓ **Q1**

```
SELECT category, SUM(amount)
FROM sales;
```

👉 ❌ Error

✓ Fix:

```
GROUP BY category
```

❓ **Q2**

```
SELECT * FROM sales WHERE SUM(amount) > 1000;
```

👉 ❌ Wrong

✓ Fix:

```
HAVING SUM(amount) > 1000;
```

❓ **Q3**

Difference:

COUNT(*) vs COUNT(column)

👉 Key:

- COUNT(*) → counts all rows
- COUNT(column) → ignores NULL

🔥 **COMPLEX SCENARIOS (Think Like Data Engineer)**

🧩 **Scenario 1: Multi-level Aggregation**

👉 First aggregate per customer
👉 Then aggregate again

```
SELECT AVG(total_spent)
FROM (
  SELECT customer_id, SUM(amount) AS total_spent
  FROM sales
  GROUP BY customer_id
) t;
```

🧩 **Scenario 2: Conditional Aggregation**

```
SELECT
  SUM(CASE WHEN category = 'Electronics' THEN amount ELSE 0 END) AS electronics_sales,
  SUM(CASE WHEN category = 'Furniture' THEN amount ELSE 0 END) AS furniture_sales
FROM sales;
```

🧩 **Scenario 3: Count unique per group**

```
SELECT store_location, COUNT(DISTINCT customer_id)
FROM sales
GROUP BY store_location;
```

🧠 **FINAL MENTAL MODEL**

When you see aggregation, always ask:

1. **What is the grouping level?**
2. **What is being aggregated?**
3. **Do I need filtering before or after aggregation?**

🚀 **Straight Advice**

Don't just read these.

👉 Try:

- Modify queries
- Break them
- Fix them

That’s how you become strong.